Chapter 5 - Monte Carlo Methods (Introduction)

Monte Carlo (MC) methods learn value functions and optimal policies **from experience** instead of requiring a complete model of the environment.

Characteristics

- No knowledge of transition probabilities is required.
- Uses real or simulated experience.
- Estimates values by averaging **complete returns**.
- Requires **episodic tasks** (episodes must terminate).
- Updates values and policies **only after an episode ends**.
- MC methods can be incremental on an episode-by-episode basis, but not step-by-step (online) within an episode.

Key Differences Between DP and MC

Dynamic Programming	Monte Carlo
Requires a complete model	Does not require a model
Uses transition and reward probabilities	Uses sampled episodes
Computes expected values	Averages observed returns
Does not require complete episodes	Updates after complete episodes

Concepts

- **Experience:** Sequences of states, actions, and rewards.
- **Return (G):** Total discounted reward from a state until episode termination.
- **Value Estimation:** Average of observed returns.
- **Sample-Based Learning:** Learns from sampled episodes instead of exact probabilities.

Relation to Bandits

Bandits: - Average rewards for each action.

Monte Carlo: - Average returns for each state-action pair. Monte Carlo estimates state values by averaging many observed returns.

Monte Carlo estimates state values by averaging many observed returns.

Example:

Returns observed from state **S**:

```
1
0
1
```

```
1
0
1
```

Estimated value:

```
V(S) = Average(Returns)
     = (1+0+1+1+0+1)/6
     = 0.67
```

Nonstationarity

The return from an early state depends on future actions within the same episode, making learning nonstationary.

General Policy Iteration (GPI)

Monte Carlo follows the same GPI framework as Dynamic Programming: Two processes interact continuously:

- **Policy Evaluation:** Estimate state values from sampled returns.
- **Policy Improvement:** Make the policy greedy with respect to the current value estimates.

Repeating these steps gradually leads toward the optimal policy.

Learning Tasks

- **Prediction:** Estimate v_π and q_π for a fixed policy.
- **Control:** Find the optimal policy using repeated evaluation and improvement.

5.1 Monte Carlo Prediction

Goal

Estimate the state-value function $v_\pi(s)$ for a fixed policy using sampled episodes.

- Estimate a state's value by averaging observed returns after visiting that state.
- As more samples are collected, the estimate converges to the expected return.

Monte Carlo Prediction Methods

First-Visit MC

- Uses only the **first occurrence** of a state in each episode.
- Most commonly studied and used in this chapter.

Every-Visit MC

- Uses **every occurrence** of a state.
- Similar practical performance with slightly different theoretical properties.

First-Visit MC Algorithm

Input: A policy π to be evaluated.

1. Initialize $V(s)$ arbitrarily for every state.
2. Initialize `Returns(s)` as an empty list for every state.
3. Generate a complete episode by following π:

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$$

4. Set:

$$G = 0$$

5. Process the episode backward from $t = T - 1$ to $t = 0$:

$$G \leftarrow \gamma G + R_{t+1}$$

6. If S_t did not appear before time t in the episode:
   - Append G to `Returns(S_t)`.
   - Update:

$$V(S_t) \leftarrow \text{average}(\text{Returns}(S_t))$$

Every-Visit MC uses the same algorithm but removes the check for whether S_t appeared earlier.

Convergence

- Both **First-Visit MC** and **Every-Visit MC** converge to $ v_□(s) $ as the number of relevant visits to state $ s $ goes to infinity.
- In both methods, the goal is to estimate:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

where (G_t) is the return following time (t).

First-Visit MC

- In **First-Visit MC**, only the **first occurrence** of state (s) in each episode is used.

- The returns collected from these first visits are **independent and identically distributed (i.i.d.)**, assuming episodes are independently generated under policy $ □$.
- Each sampled return has:

$$\mathbb{E}[G \mid S_t = s] = v_\pi(s)$$

- By the **Law of Large Numbers (LLN)**, the average of these returns converges to the true value:

$$\hat{V}_n(s) \to v_\pi(s) \qquad \text{as } n \to \infty$$

- If the return variance is finite, then:

$$\text{Var}(\hat{V}_n(s)) = \frac{\sigma^2}{n}$$

so the standard deviation of the estimation error decreases as:

$$O\left(\frac{1}{\sqrt{n}}\right)$$

which means improving accuracy significantly requires many more samples.

Monte Carlo vs Dynamic Programming

Dynamic Programming	Monte Carlo
Requires transition probabilities	Uses sampled episodes only
Bootstraps	No bootstrapping
One-step backups	Full-episode backups

Backup Diagram

Monte Carlo updates a state using the **complete sampled trajectory** until the terminal state.

Advantages

- Model-free learning.
- Easy to use with simulated experience.
- Can estimate values of selected states only.
- Suitable when environment dynamics are difficult to compute.

5.2 Monte Carlo Estimation of Action Values

Motivation

Without an environment model, **state values are not enough** for choosing actions. Therefore, Monte Carlo methods estimate **action-value functions**.

Action-Value Function

$$q_\pi(s,a) = E[G_t \mid S_t = s,\ A_t = a]$$

Expected return after taking action **a** in state **s**, then following policy **π**.

Monte Carlo Estimation

The procedure is identical to state-value estimation, except updates are performed for **state–action pairs**.

First-Visit MC

- Uses the return after the **first occurrence** of a state–action pair in each episode.

Every-Visit MC

- Uses returns after **every occurrence** of the state–action pair.

Both converge to the true action value as the number of visits approaches infinity.

Problem

With a **deterministic policy**, only one action is selected in each state.

Consequences: - Other actions may never be visited. - Their Q-values cannot be estimated. - Policy improvement becomes impossible because actions cannot be compared.

Solution 1: Exploring Starts (ES)

Assumption: - Every episode starts from a randomly selected **state–action pair**. - Every state–action pair has a **nonzero probability** of being selected.

Advantage: - Guarantees every state–action pair is visited infinitely often. - Ensures convergence of Monte Carlo action-value estimates.

Limitation

Exploring Starts is **unrealistic** in real-world environments because we usually cannot control the initial state and action of every episode.

Practical Solution (Introduced Later)

Instead of Exploring Starts, use **stochastic policies**.

Properties: - Every action has a nonzero probability of being selected. - Exploration occurs naturally during learning. - More practical for real interaction with the environment.

5.3 Monte Carlo Control

How Monte Carlo estimation can be used in control, that is, to approximate optimal policies.

Generalized Policy Iteration (GPI)

Monte Carlo Control follows the GPI framework:

1. Policy Evaluation
2. Policy Improvement
3. Repeat until convergence

Both the policy and the value function improve together.

Monte Carlo Policy Iteration

$$\pi_0 \to q_{\pi_0} \to \pi_1 \to q_{\pi_1} \to \cdots \to \pi^* \to q^*$$

Policy Improvement

Construct a greedy policy from the current action-value function:

$$\pi(s) = \arg\max_a Q(s,a)$$

Since Q-values are directly estimated, **no environment model is required**.

Policy Improvement Theorem

A greedy policy is always:

- Better than the previous policy, or
- Equally good (already optimal).

Thus repeated evaluation and improvement converge toward the optimal policy.

Two Assumptions

1. Exploring Starts (ES)

Every state–action pair has a nonzero probability of being the starting pair.

Purpose: - Guarantees sufficient exploration.

---

2. Infinite Episodes

Each policy evaluation assumes infinitely many episodes.

Purpose: - Allows exact estimation of Q_π.

Problem: - Impossible in practice.

Removing the Infinite-Episode Assumption

Approach 1

Perform almost-complete policy evaluation.

Advantage - Strong theoretical guarantees.

Disadvantage - Requires too many episodes.

Approach 2 (Practical GPI)

Do not wait for policy evaluation to finish.

Instead:

```
Episode
   ↓
Update Q
   ↓
Improve Policy
   ↓
```

```
Next Episode
```

Evaluation and improvement proceed together. (In DP we did the improvement after every evaluation loop)

Monte Carlo ES Algorithm

Initialization

- Initialize an arbitrary policy.
- Initialize (Q(s,a)) arbitrarily.
- Maintain a return list for every state–action pair.

Algorithm

For each episode:

1. Select a random starting state–action pair (Exploring Starts).
2. Generate a complete episode following the current policy.
3. Compute returns backward.
4. For each **first-visited** state–action pair:
    - Store the return.
    - Update $Q(s,a)$ using the average return.
5. Improve the policy greedily:

$$\pi(s) = \arg\max_a Q(s,a)$$

Repeat.

Idea

Instead of waiting for complete policy evaluation:

```
Episode
   ↓
Update Q
   ↓
Greedy Improvement
   ↓
Next Episode
```

Evaluation and improvement occur after every episode.

Why It Works

- Q-values become more accurate over time.
- Greedy improvement continuously improves the policy.
- A suboptimal policy cannot remain stable because improved Q-values would change it.
- The only stable fixed point is the optimal policy.

Practical Issue

Problem

The basic algorithm stores **all returns** for every state–action pair.

This requires large memory.

Better Solution

Maintain only:

- Visit count
- Running average

using the incremental update formula.

Advantages

- Model-free control.
- Policy improves after every episode.
- Finds optimal policies using sampled experience.

Limitations

- Requires Exploring Starts (impractical).
- Stores many returns unless incremental averaging is used.
- Formal convergence proof is still incomplete. # 5.4 Monte Carlo Control without Exploring Starts

Motivation

Monte Carlo ES requires the unrealistic **Exploring Starts (ES)** assumption.

To eliminate this assumption, the agent must **continue exploring during learning**.

Two Approaches

On-Policy

- Behavior Policy = Target Policy
- evaluate or improve the policy that is used to make decisions

Off-Policy

- Behavior Policy ≠ Target Policy
- evaluate or improve a policy different from that used to generate the data.

This section introduces **On-Policy Monte Carlo Control**.

Soft Policies

A soft policy assigns **nonzero probability to every action**:

$$\pi(a|s) > 0$$

for every state and action.

This guarantees continual exploration.

ε-Soft Policy

A policy is ε-soft if

$$\pi(a|s) \geq \frac{\varepsilon}{|A(s)|}$$

for every state and action.

ε-Greedy Policy

Choose the greedy action with probability

$$1 - \varepsilon + \frac{\varepsilon}{|A(s)|}$$

Choose every nongreedy action with probability

$$\frac{\varepsilon}{|A(s)|}$$

ε-greedy is the ε-soft policy closest to a deterministic greedy policy.

On-Policy First-Visit MC Control

Initialize:

- Arbitrary ε-soft policy
- Arbitrary Q(s,a)
- Empty return list

Repeat for every episode:

1. Generate an episode using the current ε-soft policy.
2. Compute returns backward.
3. For each first-visited state–action pair:
    - Update Q using the average return.
4. Find

$$A^* = \arg\max_a Q(s,a)$$

5. Update the policy to ε-greedy:

$$\pi(a|s) = \begin{cases} 1-\varepsilon+\dfrac{\varepsilon}{|A(s)|}, & a = A^* \\ \dfrac{\varepsilon}{|A(s)|}, & a \neq A^* \end{cases}$$

Repeat forever.

Why Not Fully Greedy?

Without Exploring Starts,

a fully greedy policy may stop selecting some actions.

Those actions would never be evaluated again.

Therefore exploration would disappear.

Instead, we move only **toward** greediness by using ε-greedy policies.

Policy Improvement

The policy improvement theorem still applies. We proved that $\pi' \geq \pi$. The decomposition of the improvement shows:

$$\sum_a \pi'(a|s) q_\pi(s,a) = \frac{\varepsilon}{|A(s)|} \sum_a q_\pi(s,a) - \frac{\varepsilon}{|A(s)|} \sum_a q_\pi(s,a) + \sum_a \pi(a|s) q_\pi(s,a) = v_\pi(s)$$

The first two terms cancel out, proving $v_{\pi'}(s) \geq v_\pi(s)$.

Equality Case & Proof Idea (Modified Environment)

If $\pi' = \pi$ (no further improvement), then π is already optimal **among all ε-soft policies**.

The proof constructs a **modified environment** to handle the ε-soft constraint mathematically.

1. The Modified Environment

Instead of restricting the policy, we move the randomness **inside the environment**: - With probability $1-\varepsilon$: The environment behaves normally. - With probability ε: The environment overrides the action with a random one.

This implies an **altered transition probability**:

$$\tilde{p}(s', r|s,a) = (1-\varepsilon) p(s', r|s,a) + \frac{\varepsilon}{|A(s)|} \sum_{a'} p(s', r|s, a')$$

2. Bellman Optimality in New Environment

The optimal value function $\tilde{v}_*$ in this new environment satisfies:

$$\tilde{v}_*(s) = \max_a \sum_{s',r} \left[(1-\varepsilon) p(s', r|s,a) + \sum_{a'} \frac{\varepsilon}{|A(s)|} p(s', r|s, a') \right] [r + \gamma \tilde{v}_*(s')]$$

3. Proof of Equality

When $\pi' = \pi$, we satisfy:

$$v_\pi(s) = (1-\varepsilon) \max_a q_\pi(s,a) + \frac{\varepsilon}{|A(s)|} \sum_a q_\pi(s,a)$$

Expanding this leads to a structure identical to the $\tilde{v}_*$ equation. Because $\tilde{v}_*$ is the **unique solution** to the Bellman optimality equation, it must be that $v_\pi = \tilde{v}_*$.

Advantages

- No Exploring Starts required.
- Exploration is maintained automatically.
- Still follows the GPI framework.
- Policy improves after every episode.

Limitations

- Learns the optimal **ε-soft policy**, not necessarily the deterministic optimal policy.
- Exact policy evaluation is still assumed during the theoretical proof.
- If ε is fixed, some random exploration always remains.

Comparison

Monte Carlo ES	On-Policy MC
Requires Exploring Starts	No Exploring Starts
Greedy improvement	ε-greedy improvement
Exploration from initial state	Exploration through action probabilities
Finds deterministic optimal policy (under ES assumptions)	Finds the optimal ε-soft policy

5.5: Off-policy Prediction via Importance Sampling

One of the biggest challenges in reinforcement learning is balancing **exploration** and **exploitation**.

- To learn the optimal policy, the agent must explore different actions.
- However, the value we ultimately want corresponds to the **optimal policy**, not the exploratory one.

Off-policy learning solves this problem by using **two different policies**.

Two Policies

1. Behavior Policy (b)

- Generates the data (episodes).
- Usually exploratory.

- Often ε-greedy or random.

2. Target Policy (π)

- The policy whose value we want to estimate.
- Can be deterministic.
- Usually becomes the optimal greedy policy.

The agent **acts according to the behavior policy** but **learns about the target policy**.

On-policy vs Off-policy

On-policy	Off-policy
One policy	Two policies
Learn the same policy used for acting	Learn a different policy from the one used for acting
Simpler	More general and powerful
Lower variance	Higher variance
Faster convergence	Usually slower

Advantages of Off-policy Learning

- Learn from exploratory behavior.
- Learn from demonstrations by humans.
- Learn from fixed datasets collected previously.
- Useful for model learning and offline reinforcement learning.

Prediction Problem

In this section both policies are **fixed**.

Goal:

Estimate

- $v_\pi(s)$
- $q_\pi(s,a)$

using episodes generated by another policy b.

Coverage Assumption

Off-policy learning requires

$$\pi(a|s) > 0 \Rightarrow b(a|s) > 0$$

Meaning:

If the target policy can choose an action, the behavior policy must also choose that action with non-zero probability.

Otherwise, there will be no data for estimating that action.

Consequences:

- Behavior policy must usually be stochastic.
- Target policy may be deterministic.

Importance Sampling

Returns are collected under the behavior policy.

Their expectation is

$$E[G_t|S_t = s] = v_b(s)$$

instead of

$$v_\pi(s)$$

Importance Sampling corrects this distribution mismatch.

Importance Sampling Ratio

The trajectory probability ratio is

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

Properties:

- Transition probabilities cancel out.
- No model of the environment is needed.
- Depends only on:

- target policy
- behavior policy
- observed trajectory

Key Property

Importance sampling guarantees

$$E[\rho G] = v_\pi(s)$$

allowing off-policy returns to estimate the target policy value.

Ordinary Importance Sampling

Estimator

$$V(s) = \frac{1}{N} \sum \rho G$$

Advantages

- Unbiased estimator.

Disadvantages

- Very high variance.
- Variance may become infinite.
- Learning can be unstable.

Weighted Importance Sampling

Estimator

$$V(s) = \frac{\sum \rho G}{\sum \rho}$$

Advantages

- Much lower variance.
- More stable.
- Preferred in practice.

Disadvantages

- Biased.
- Bias approaches zero as the number of samples increases.

Ordinary vs Weighted Importance Sampling

Ordinary IS	Weighted IS
Unbiased	Biased
High variance	Low variance
May have infinite variance	Variance remains bounded
Better theoretical property	Better practical performance

First-Visit vs Every-Visit

First-Visit

- Uses only the first occurrence of each state.
- Ordinary IS is unbiased.
- Weighted IS is biased.

Every-Visit

- Uses every occurrence.
- Easier implementation.
- Easier extension to function approximation.
- Both estimators are biased, but the bias approaches zero asymptotically.

Incremental Implementation

Previous Monte Carlo methods stored **all returns** for each state or state-action pair and computed their average.

This is memory inefficient.

Instead, Monte Carlo can be implemented **incrementally**, updating estimates after each episode without storing past returns.

On-Policy Monte Carlo

For on-policy prediction, the same incremental sample-average update from Chapter 2 is used.

Instead of averaging rewards, we average returns.

Update rule:

$$V_{n+1} = V_n + \frac{1}{n}(G_n - V_n)$$

No additional memory is required besides the current estimate and visit count.

Off-Policy Monte Carlo

There are two cases.

1. Ordinary Importance Sampling

Returns are first multiplied by the importance-sampling ratio:

$$\rho G$$

Then the standard incremental averaging formula is applied.

No new algorithm is needed.

---

2. Weighted Importance Sampling

Weighted importance sampling estimates

$$V = \frac{\sum WG}{\sum W}$$

This requires a different incremental update.

Cumulative Weight

Maintain

$$C_n = \sum_{i=1}^{n} W_i$$

where (C_n) is the cumulative sum of importance weights.

Update:

$$C_{n+1} = C_n + W_{n+1}$$

Incremental Weighted Update

Update the estimate using

$$V_{n+1} = V_n + \frac{W_n}{C_n}(G_n - V_n)$$

A sample with a larger importance weight has a greater influence on the estimate.

Incremental Off-Policy MC Algorithm

For each state-action pair:

Initialize:

- $Q(s,a)$
- $C(s,a) = 0$

For each episode generated by behavior policy b:

1. Traverse the episode backward.
2. Compute the return G.
3. Update cumulative weight:

$$C(s,a) \leftarrow C(s,a) + W$$

4. Update action value:

$$Q(s,a) \leftarrow Q(s,a) + \frac{W}{C(s,a)}(G - Q(s,a))$$

5. Update importance weight:

$$W \leftarrow W \times \frac{\pi(a|s)}{b(a|s)}$$

On-Policy as a Special Case

If

$$\pi = b$$

then

$$\frac{\pi(a|s)}{b(a|s)} = 1$$

so

$$W = 1$$

for every step.

Thus, the same algorithm naturally reduces to standard on-policy Monte Carlo prediction.

Why Traverse Backward?

The importance-sampling ratio is a product over future actions.

Traversing backward allows the ratio to be updated incrementally by multiplying one additional policy ratio at each step, avoiding repeated computation.

5.7: Off-policy Monte Carlo Control

Off-policy Monte Carlo Control extends off-policy prediction to the **control problem**.

Instead of only evaluating a fixed policy, the algorithm **learns the optimal action-value function** q^* while simultaneously improving the target policy.

The key idea is to **separate acting from learning**:

- The **behavior policy** generates experience.
- The **target policy** is evaluated and improved.

Behavior Policy vs Target Policy

Behavior Policy (b)

- Generates episodes.
- Must be **soft**, meaning every action has a non-zero probability.
- Usually an ε-soft or ε-greedy policy.
- Responsible for exploration.

Target Policy (π)

- Learned from the collected experience.
- Always greedy with respect to the current action-value estimates.
- Gradually converges to the optimal policy.

Coverage Requirement

The behavior policy must satisfy the **coverage assumption**.

If the target policy can choose an action,

$$\pi(a|s) > 0$$

then the behavior policy must also occasionally choose it:

$$b(a|s) > 0$$

Otherwise, some actions would never be observed and their values could never be learned.

Algorithm

For each episode:

1. Generate an episode using the behavior policy b.
2. Initialize

$$G = 0, \qquad W = 1$$

3. Process the episode backward.

For every state-action pair:

Update the return

$$G \leftarrow \gamma G + R_{t+1}$$

Update the cumulative weight

$$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$$

Update the action-value estimate

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} \left(G - Q(S_t, A_t)\right)$$

Improve the target policy

$$\pi(S_t) = \arg\max_a Q(S_t, a)$$

Early Termination

After updating the target policy, the algorithm checks

```
If At ≠ π(St)

    Stop processing the episode
```

This happens because the target policy is **deterministic and greedy**.

If the behavior policy takes a nongreedy action,

then

$$\pi(A_t|S_t) = 0.$$

Therefore, the importance-sampling ratio becomes zero.

Consequently, all earlier state-action pairs receive zero weight, so there is no reason to continue processing the episode.

Importance Sampling Weight

If the selected action agrees with the target policy,

the importance weight is updated as

$$W \leftarrow W \times \frac{1}{b(A_t|S_t)}.$$

Since the target policy is deterministic,

$$\pi(A_t|S_t) = 1,$$

the usual importance-sampling ratio

$$\frac{\pi(A_t|S_t)}{b(A_t|S_t)}$$

simplifies to

$$\frac{1}{b(A_t|S_t)}.$$

Why the Behavior Policy Must Be Soft

The target policy eventually becomes greedy.

Without exploration,

many actions would never be sampled.

A soft behavior policy guarantees that every state-action pair is visited infinitely often, ensuring convergence to the optimal policy.

Limitation of the Algorithm

A major weakness is that learning occurs **only from the tail of an episode**.

As soon as a nongreedy action appears,

the backward update stops.

Therefore,

- Long episodes may contribute only a few updates.
- Early states are updated less frequently.
- Learning can become very slow.

This problem becomes more severe when nongreedy actions occur frequently.

Possible Improvements

The book suggests two possible solutions:

- **Temporal-Difference (TD) learning**, introduced in the next chapter, which updates values after every step instead of waiting until the end of an episode.
- Using a discount factor

$$\gamma < 1,$$

which reduces the influence of distant future rewards.

5.8: Discounting-aware Importance Sampling

The Problem

Recall that the return is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T.$$

Ordinary Importance Sampling multiplies the entire return by

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}.$$

When episodes are long, this ratio becomes a product of many probabilities.

For example,

- Episode length = 100
- Discount factor

$$\gamma = 0$$

Then

$$G_0 = R_1,$$

because all future rewards disappear.

However, Ordinary Importance Sampling still multiplies the return by

$$\frac{\pi(A_0|S_0)}{b(A_0|S_0)} \cdot \frac{\pi(A_1|S_1)}{b(A_1|S_1)} \cdots \frac{\pi(A_{99}|S_{99})}{b(A_{99}|S_{99})}.$$

Only the **first ratio** actually affects the reward.

The remaining 99 ratios only increase variance without changing the expected value.

Main Idea

Instead of treating the return as one large quantity,

the return is decomposed into several **partial returns**.

Each partial return ends at a different horizon.

These are called **Flat Partial Returns**.

Flat Partial Return

A flat partial return is

$$\bar{G}_{t:h} = R_{t+1} + R_{t+2} + \cdots + R_h, \qquad t < h \leq T.$$

Unlike the normal return,

there is **no discounting inside the sum**.

Rewriting the Return

The standard discounted return can be written as

$$G_t = (1-\gamma)\bar{G}_{t:t+1} + (1-\gamma)\gamma\bar{G}_{t:t+2} + (1-\gamma)\gamma^2\bar{G}_{t:t+3} + \cdots + \gamma^{T-t-1}\bar{G}_{t:T}.$$

Therefore,

the original return is simply a weighted sum of many flat partial returns.

Truncated Importance Sampling

Each partial return only depends on rewards up to horizon h.

Therefore,

it only needs the importance-sampling ratio up to that horizon.

Instead of

$$\rho_{t:T-1},$$

we use

$$\rho_{t:h-1}.$$

This greatly reduces the number of unnecessary probability ratios.

Discounting-aware Ordinary Importance Sampling

The estimator becomes

$$V(s) = \frac{\sum \left[(1-\gamma)\sum_h \gamma^{h-t-1}\rho_{t:h-1}\bar{G}_{t:h} + \gamma^{T-t-1}\rho_{t:T-1}\bar{G}_{t:T}\right]}{|T(s)|}.$$

Instead of weighting one long return,
the algorithm weights many shorter partial returns.

Discounting-aware Weighted Importance Sampling

The weighted estimator becomes

$$V(s) = \frac{\sum \left[(1-\gamma)\sum_h \gamma^{h-t-1}\rho_{t:h-1}\bar{G}_{t:h} + \gamma^{T-t-1}\rho_{t:T-1}\bar{G}_{t:T}\right]}{\sum \left[(1-\gamma)\sum_h \gamma^{h-t-1}\rho_{t:h-1} + \gamma^{T-t-1}\rho_{t:T-1}\right]}.$$

This is the weighted version of the same idea.

Why Does This Reduce Variance?

Suppose

$$\gamma = 0.$$

Then

$$G_0 = R_1.$$

Only the first action affects the return.

The remaining importance ratios are completely unnecessary.

Instead of multiplying by

$$100$$

probability ratios,

we only multiply by

$$\frac{\pi(A_0|S_0)}{b(A_0|S_0)}.$$

Removing unnecessary ratios dramatically lowers variance.

Special Case

If

$$\gamma = 1,$$

then no discounting exists.

The new estimator becomes exactly the same as the ordinary off-policy estimator from Section 5.5.

Therefore,

Discounting-aware Importance Sampling is only useful when

$$\gamma < 1.$$

Advantages

- Uses the structure of discounted returns.
- Removes unnecessary importance-sampling ratios.
- Significantly reduces variance.
- Produces more stable off-policy estimates.
- Especially effective for long episodes.

- Very beneficial when the discount factor is much smaller than one.

Limitations

- More complicated than ordinary importance sampling.
- Provides no benefit when

$$\gamma = 1.$$

- Mainly useful for discounted tasks.

5.9: Per-decision Importance Sampling

Per-decision Importance Sampling improves ordinary importance sampling by applying the importance-sampling ratio **separately to each reward**, instead of multiplying the entire return by one large ratio.

The key observation is that each reward depends only on the decisions made **before that reward**, not on future decisions.

Motivation

Ordinary importance sampling computes

$$\rho_{t:T-1} G_t$$

where

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Every reward is multiplied by the **same full importance-sampling ratio**, even though later actions cannot affect earlier rewards.

This introduces unnecessary variance.

Key Observation

The expected value of future importance-sampling factors is

$$E\left[\frac{\pi(A_k|S_k)}{b(A_k|S_k)}\right] = 1.$$

Therefore, these future factors do not change the expectation but only increase the variance.

Truncated Importance Ratios

Instead of using one full ratio,

each reward uses only the importance ratio up to the decision that produced it.

Examples:

For the first reward:

$$R_{t+1} \to \rho_{t:t}$$

For the second reward:

$$R_{t+2} \to \rho_{t:t+1}$$

For the third reward:

$$R_{t+3} \to \rho_{t:t+2}$$

Per-decision Return

The modified return is

$$\tilde{G}_t = \rho_{t:t}R_{t+1} + \gamma\rho_{t:t+1}R_{t+2} + \gamma^2\rho_{t:t+2}R_{t+3} + \cdots + \gamma^{T-t-1}\rho_{t:T-1}R_T.$$

Value Estimator

The value estimate becomes

$$V(s) = \frac{\sum_{t\in T(s)}\tilde{G}_t}{|T(s)|}.$$

Advantages

- Lower variance than ordinary importance sampling.
- Keeps the same expected value (unbiased in the first-visit case).
- Removes unnecessary future importance-sampling factors.
- Works even when

$$\gamma = 1$$

.

Limitation

A consistent weighted version of Per-decision Importance Sampling has not yet been established.
